Principal Author: *u/Significant-Wrap-589*
Co-Author & Document Designer: *u/MurkyUnit3180*

LINUX

A BRIEF GUIDE TO GAMING

A community guide by r/Indiangamers

Version 1.0 · May 2026 · Licensed under

This guide is a living document and will be updated over time.

Contents

1. What is Linux?

At its most precise, Linux is not an operating system; it is a kernel. The *kernel* is the core program that sits between your hardware and everything else running on your machine. It manages the CPU, memory, storage, and peripheral devices. As the GNU Project puts it:

> *"Linux is the kernel: the program in the system that allocates the machine's resources to the other programs that you run. The kernel is an essential part of an operating system, but useless by itself."*
>
> — GNU Project, gnu.org

What most people call "Linux" is more accurately a **GNU/Linux distribution**, the kernel bundled with system software, a package manager, a desktop environment, and various tools that make it a complete, usable OS.

According to kernel.org, Linux was originally written from scratch by Linus Torvalds as a Unix-like system, targeting POSIX compliance, with support for true multitasking, virtual memory, and shared libraries. It was first developed for 32-bit x86 PCs but today runs on virtually every processor architecture in existence.

1.1. How a Linux System Is Structured

A complete Linux system consists of several layers:

- **Bootloader**: loads the OS on startup (e.g. GRUB)
- **Kernel**: manages hardware, memory, and processes
- **Init system**: bootstraps user space after boot (typically `systemd`)
- **Shell**: your interface to the system (e.g. `bash`, `zsh`)
- **Desktop Environment**: the graphical layer (e.g. GNOME, KDE Plasma)
- **Package Manager**: installs and manages software

Kernel ≠ OS

If someone hands you "the Linux kernel", you cannot use it as a desktop system. You need a full *distribution*, which packages the kernel with everything else. When people say they "run Linux", they mean a distribution like Ubuntu, Arch, or Fedora.

1.2. What is a Distribution?

A **distribution** (or *distro*) is a complete operating system built around the Linux kernel. Different distros make different choices about which desktop environment, package manager, and software to include. The kernel itself is the same, only what surrounds it differs. Common package managers you will encounter:

- `apt`: Debian, Ubuntu, Pop!_OS
- `dnf`: Fedora
- `pacman`: Arch Linux, Manjaro

To update your system, the command looks like this depending on your distro:

```bash
# Debian/Ubuntu
sudo apt update && sudo apt upgrade

# Fedora
sudo dnf upgrade

# Arch
sudo pacman -Syu

```

Keeping your system updated is not optional, it is the foundation of a stable gaming setup. On Linux, drivers, compatibility layers like *Proton*, and core libraries like *glibc* and *Mesa* all ship through the package manager. An outdated system means outdated graphics drivers, which directly translates to lower performance, crashes, or games outright refusing to launch.

Unlike Windows, where driver updates are fragmented across manufacturer websites and Windows Update, Linux consolidates everything into a single command. Your GPU driver, kernel, and game runtime dependencies all update together.

Warning

Never run a partial upgrade. On Arch, `pacman -Sy package` without a full `-Syu` first can break your system by introducing dependency mismatches.

1.3. Why Linux Matters

Linux may occupy a modest share of consumer desktops, but it quietly runs the world. *Every* major cloud provider (AWS, Google Cloud, and Microsoft Azure) runs Linux as their foundational operating system. As of 2025, Linux powers 90% of public cloud infrastructure, 100% of the world's top 500 supercomputers, and 77% of all web server deployments worldwide.

By the Numbers (2025)

- **90%** of public cloud infrastructure runs Linux
- **100%** of TOP500 supercomputers run Linux
- **78.5%** of developers use Linux as a primary or secondary platform
- **96.4%** of production Kubernetes clusters run on Linux

The phone in your pocket almost certainly runs Android, which is built on the Linux kernel. The servers that stream your games, process your payments, and host your favourite websites run Linux. It is certainly the backbone of modern computing.

For the end user, what this means is stability, transparency, and control. Linux is free, open-source, and maintained by thousands of contributors worldwide. No forced telemetry or licence fees.

1.4. Linux and Gaming

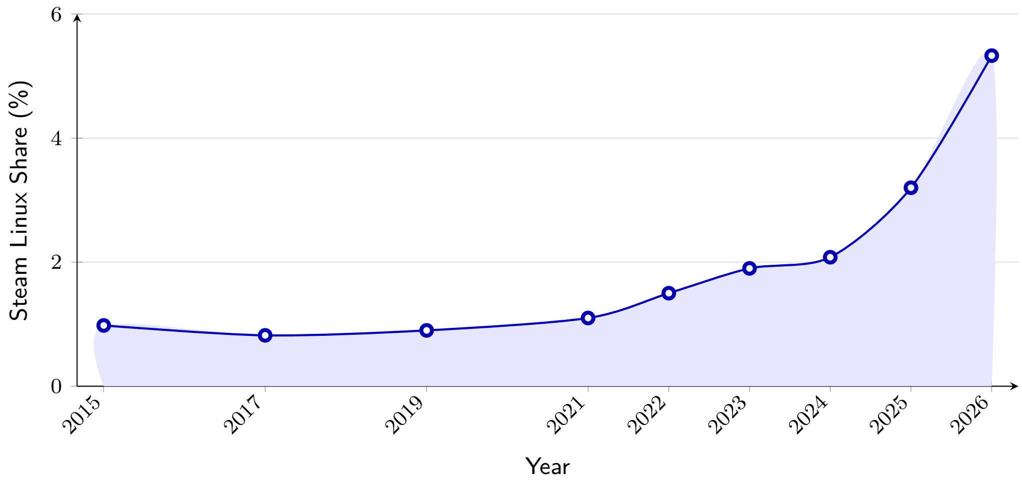
Linux gaming has undergone a fundamental shift since Valve released *Proton* in 2018. Proton is a compatibility layer developed by Valve in collaboration with CodeWeavers, built on top of *Wine*, that translates Windows API calls into Linux-compatible equivalents. In plain terms: it lets you run Windows games on Linux with little to no configuration.

The release of the Steam Deck in 2022 accelerated this significantly. Running Valve's own *SteamOS*, a Linux-based operating system, the Steam Deck proved that Linux could run a serious gaming library on consumer hardware. As of November 2025, *Proton 10.0-3* is the latest stable release, bringing fixes and expanded support across thousands of titles including *God of War Ragnarök*, *Marvel's Spider-Man Remastered*, and more.

How Proton Works

A Windows game makes calls to **DirectX** and the **Windows API**. *Proton* intercepts these calls and translates them in real time using **DXVK** (DirectX → Vulkan) and **Wine** (Windows API → Linux). The game never knows it is not running on Windows.

Linux Share on Steam (Valve Hardware Survey)



Source: Valve Steam Hardware & Software Survey

That said, Linux gaming is not without its caveats. Games with aggressive kernel-level anti-cheat (such as Valorant's Vanguard or PUBG's BattlEye in certain configurations) remain the primary blocker. This is an active area of development, and compatibility improves with every *Proton* release.

Anti-Cheat Warning

Before switching, check your most-played games on ProtonDB and areweanticheatyet.com. Kernel-level anti-cheat is the one area where Linux genuinely cannot help you, yet.

2. Installing Linux

This guide uses *CachyOS* as the recommended distribution. CachyOS is an Arch-based rolling-release distribution that bundles all necessary gaming libraries, Proton, and dependencies into a single meta-package install, making it one of the most convenient starting points for Linux gaming without sacrificing access to the full Arch ecosystem.

> **Why Not Ubuntu or Fedora?**
>
> Both are solid distros. CachyOS is chosen here purely for its convenience — one meta-package gets you a complete gaming setup. That is the only reason.

2.1. Choosing a Distro

With hundreds of Linux distributions available, the choice can feel overwhelming. For this guide, the decision has already been made: **CachyOS**. This subsection explains why, and briefly covers the alternatives worth knowing about.

2.1.1. Why CachyOS?

CachyOS is an Arch-based rolling-release distribution. Its primary advantage for this guide is convenience: gaming libraries, Proton, and all necessary dependencies are available as a single meta-package, meaning the gap between a fresh install and a working gaming setup is minimal.

Being Arch-based is also a practical advantage: Valve built SteamOS on Arch, meaning kernel-level fixes and gaming improvements from Valve tend to trickle down to Arch-based distros like CachyOS naturally.

2.1.2. Alternatives Worth Knowing

CachyOS is not the only valid choice. The following are the most relevant alternatives:

- **Bazzite**: Fedora-based, immutable, excellent out-of-the-box gaming experience. Recommended for users who want something that simply works with minimal configuration. The safer choice for beginners.
- **Nobara**: Fedora-based, created by the developer of Proton-GE. Gaming tools pre-installed, good hardware support.
- **EndeavourOS**: Arch-based, minimal, close to vanilla Arch but with a friendlier installer. Less opinionated than CachyOS.
- **Ubuntu / Linux Mint**: Debian-based, highly stable, large community. Not gaming-optimized, but reliable and well-documented.

> **Rolling Release vs. Stable Release**
>
> CachyOS is a **rolling release**, packages update continuously rather than in fixed version cycles. This means you always have the latest drivers and software, which matters for gaming. The trade-off is that updates occasionally introduce breakage. This guide covers basic troubleshooting for exactly that reason.

2.2. Preparing the Installation Medium

Before Linux can be installed, a bootable installation medium must exist. Traditional tools such as *Rufus* or *balenaEtcher* flash a single ISO directly onto a USB drive; which works, but comes with a significant limitation: every time a different ISO is needed, the drive must be reformatted and rewritten from scratch.

This guide takes a different approach. Since a fallback Windows installer is useful to have on hand, in case anything goes wrong, both the CachyOS and Windows 11 ISOs will live on the same USB drive simultaneously. This is made possible by *Ventoy*.

Ventoy installs a small bootloader onto the USB once. After that, any number of ISO files can simply be copied onto the drive like normal files. On boot, Ventoy presents a menu and lets you choose which ISO to launch. No reflashing required.

2.2.1. Downloads

Before proceeding, download the following:

- **Ventoy** (Windows installer): ventoy.net/en/download.html
- **CachyOS Desktop Edition ISO**: cachyos.org/download
- **Windows 11 Disk Image**: microsoft.com

2.2.2. Installing Ventoy

1. Plug in your USB drive
2. Extract the downloaded `Ventoy_windows.zip` and open `Ventoy2Disk.exe`
3. Your USB drive should appear in the device list
4. Click Install, Ventoy will write its bootloader to the drive
5. Once complete, open 🗁 `This PC` in Windows File Explorer
6. Copy both ISO files directly onto the USB drive, no extraction needed

> **Warning**
>
> Installing Ventoy will erase all existing data on the USB drive. Back up anything important before proceeding. A USB drive of at least **16 GB** is recommended to fit both ISOs comfortably.

2.3. Booting into CachyOS

With the USB prepared, the next step is to tell your PC to boot from it rather than the internal drive. This is done through the *BIOS* or *UEFI* firmware settings, a low-level interface that controls hardware behaviour before the OS loads.

2.3.1. Entering the BIOS

1. Plug in the USB drive
2. Restart your PC
3. As soon as the screen turns on, repeatedly press the BIOS entry key for your device. Common keys are F2, Delete, F10, or F12, the exact key varies by manufacturer and can be found with a quick search for your device model

> **Common BIOS Entry Keys by Manufacturer**
>
> - **ASUS**: Delete or F2
> - **MSI**: Delete
> - **Gigabyte**: Delete or F2
> - **Dell**: F2 or F12
> - **HP**: F10 or Esc
> - **Lenovo**: F2 or Fn + F2

2.3.2. Configuring Boot Order

Once inside the BIOS:
1. Navigate to a tab labelled Boot, Boot Options, or Boot Priority, the exact name varies by firmware
2. Move your USB device to the top of the boot order, above the internal drive
3. Locate the Secure Boot option and set it to Disabled
4. Press F10 to save and exit. Or navigate to Exit » Save Changes and Exit

Why Disable Secure Boot?

Secure Boot is a firmware security feature that only allows signed bootloaders to run. While CachyOS supports Secure Boot, disabling it avoids potential complications during installation and is the recommended approach for a first-time Linux install. It can be re-enabled after setup if needed.

2.3.3. Selecting the CachyOS ISO in Ventoy

After saving BIOS settings, your PC will restart and boot into the Ventoy menu:
1. Use ↑ and ↓ to navigate to the CachyOS ISO
2. Press ↵ to boot into it
3. CachyOS will load into a live environment, nothing is installed yet

2.4. The Installation Process

With CachyOS booted from the Ventoy menu, the installation can begin. This subsection walks through every step of the Calamares installer sequentially, following the official CachyOS documentation.

2.4.1. Don't Panic; Boot Messages

As the ISO loads, you will likely see a screen full of rapidly scrolling white text on a black background. This is normal.

> *Linux briefly displays system initialisation messages before the graphical desktop starts. Unlike Windows, which conceals this process behind splash screens and animations, Linux exposes it. These are kernel and `systemd` messages logged in real time as hardware is detected and services are started.*
>
> — Behaviour described in the Arch Wiki: systemd/Journal and Debian Reference, Chapter 3: System Initialisation

What you are seeing is `systemd`, the init system responsible for bootstrapping the user space after the kernel loads. When the kernel executes `systemd` as process 1, it takes over system initialisation, orchestrating service startup, dependency tracking, and system management. Every line of text is a service starting, a device being detected, or a filesystem being mounted. It looks chaotic. It is not.

The messages are produced by `systemd-journald`, the journal service that logs both kernel and system messages, either into persistent binary data below `/var/log/journal` or into volatile binary data below `/run/log/journal/`. Once installed, you can review any boot's messages at any time with:

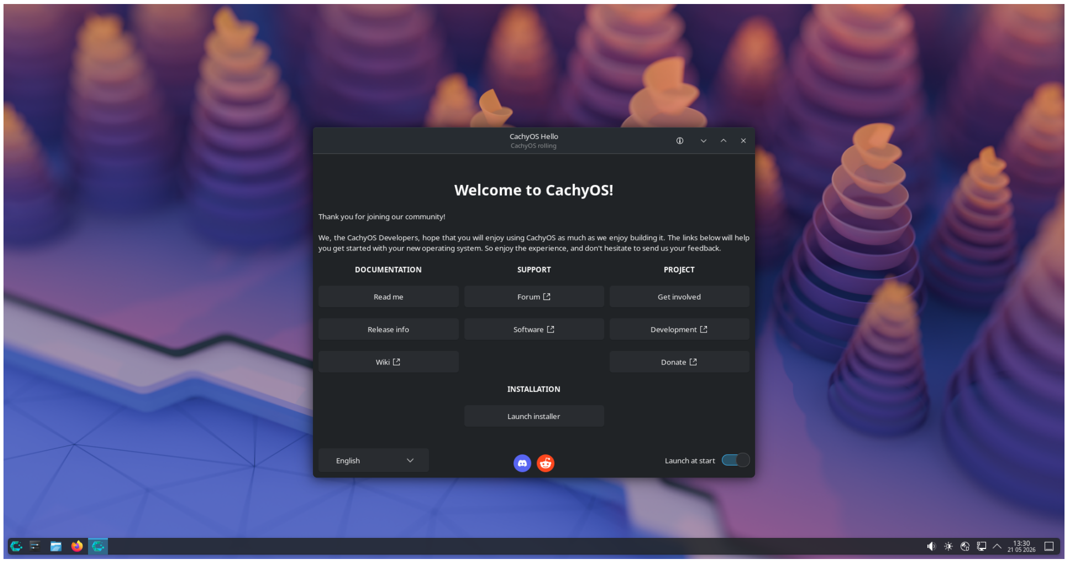
```
journalctl -b     # current boot messages
journalctl -b -1  # previous boot messages
```

```
Starting Apply Kernel Variables...
[  OK  ] Mounted Temporary /etc/pacman.d/gnupg directory.
[  OK  ] Mounted Temporary directory /tmp.
[    20.480467    ] vmwgfx 0000:00:02.0: [drm] *ERROR* vmwgfx seems to be running on an
    unsupported hypervisor.
[    20.480475    ] vmwgfx 0000:00:02.0: [drm] *ERROR* This configuration is likely broken.
[    20.480478    ] vmwgfx 0000:00:02.0: [drm] *ERROR* Please switch to a supported graphics
    device to avoid problems.
[  OK  ] Finished Apply Kernel Variables.
         Starting Network Management...
         Starting Network Name Resolution...
[  OK  ] Started Network Management.
         Starting Enable Persistent Storage in systemd-networkd...
         Starting Wait for Network to be Online...
[  OK  ] Started Network Name Resolution.
[  OK  ] Reached target Host and Network Name Lookups.
[  OK  ] Listening on Load/Save RF Kill Switch Status /dev/rfkill Watch.
         Starting Virtual Console Setup...
[  OK  ] Stopped Virtual Console Setup.
         Starting Virtual Console Setup...
[  OK  ] Finished Enable Persistent Storage in systemd-networkd.
[  OK  ] Stopped Virtual Console Setup.
         Starting Virtual Console Setup...
[  OK  ] Finished Virtual Console Setup.
[  OK  ] Finished Wait for udev To Complete Device Initialization.
[  OK  ] Reached target ZFS pool import target.
         Starting Mount ZFS filesystems...
[  OK  ] Finished Mount ZFS filesystems.
[  OK  ] Reached target Local File Systems.
[  OK  ] Listening on Boot Loader Control Service Socket.
[  OK  ] Listening on System Extension Image Management.
         Starting Automatic Boot Loader Update...
         Starting Create System Files and Directories...
         Starting Load JSON user/group Records from Credentials...
[  OK  ] Finished Automatic Boot Loader Update.
[  OK  ] Finished Load JSON user/group Records from Credentials.
[  OK  ] Finished Create System Files and Directories.
         Starting Rebuild Dynamic Linker Cache...
         Starting Initial Setup...
         Starting Rebuild Journal Catalog...
         Starting Record System Boot/Shutdown in UTMP...
[  OK  ] Finished Record System Boot/Shutdown in UTMP.
[  OK  ] Finished Initial Setup.
[  OK  ] Reached target First Boot Complete.
         Starting Save Transient machine-id to Disk...
[  OK  ] Finished Rebuild Journal Catalog.
[  OK  ] Finished Save Transient machine-id to Disk.
[      ***      ] (3 of 3) A start job is running for Rebuild Dynamic Linker Cache (16s / no
    limit)
```

Prefix	Meaning
`[ OK ]`	Started successfully
`[ INFO ]`	Informational
`[ * ]`	Starting, wait

Prefix	Meaning
`[FAILED]`	Ignorable on live ISO
`[WARNING]`	Non-critical
`[DEPENDS]`	Waiting on another service

Wait for the messages to finish. The graphical desktop will load automatically. Once it does, you will be greeted by the **CachyOS Hello** window.



CachyOS live desktop with Hello window on first boot

2.4.2. Network Connectivity

Before launching the installer, a stable internet connection is required for the online installation mode, which downloads the latest packages during install.

1. Locate the **network tray icon** in the bottom-right corner of the taskbar
2. Click it — your available Wi-Fi networks will appear
3. Select your network and enter your password
4. Confirm connectivity before proceeding

Warning

Do not launch the installer on an unstable connection. If the download stalls at 33% progress, it is almost always a network issue — not a system failure. The official CachyOS FAQ confirms this is the most common installation failure point.

Once connected, click **Launch Installer** in the CachyOS Hello window. The Calamares installer will open.

2.4.3. Language

The first tab of Calamares asks for your preferred language. Select it from the list and click **Next**.

2.4.4. Timezone and Location

Select your timezone from the map or the dropdown. This sets your system clock correctly and determines locale defaults. Click **Next**.

2.4.5. Keyboard Layout

Select your keyboard layout. If you are unsure, the default for your selected region is usually correct. Use the test field at the bottom of the screen to verify. Click **Next**.

2.4.6. Bootloader

What is a Bootloader?

A bootloader is the software that runs before the operating system loads. It presents a menu where you select which OS to boot into. Windows has its own bootloader, **Windows Boot Manager**, but hides it behind splash screens. Linux exposes it directly and gives you the choice.

CachyOS offers two bootloader choices:

- **Limine** (recommended); a portable, modern bootloader and the reference implementation of the Limine boot protocol. According to the CachyOS installer changelog, Limine is now the default and ships with automatic Btrfs snapshot support out of the box
- **GRUB**, the GNU GRand Unified Bootloader, the traditional Linux standard. More compatible with dual-boot setups and older hardware, but does not offer automatic snapshot integration

Choose Limine for this guide and click **Next**.

What are Snapshots?

Snapshots are point-in-time copies of a filesystem. If a system update breaks something, you can boot a previous snapshot and restore your system to its last working state. Think of them as save states — if something goes wrong, load the previous save.

2.4.7. Partitioning

This guide assumes a full, dedicated Linux install, no dual boot. Select **Erase Disk**.

Warning

Erase Disk permanently deletes everything on the selected drive. Back up your Windows data before proceeding. Confirm you have selected the correct disk. If multiple drives are present, disconnect any you do not want wiped.

After selecting Erase Disk, you will be prompted to choose a filesystem:

- **Btrfs** (recommended); supports snapshots natively. Combined with Limine, gives you automatic rollback capability. Calamares auto-detects whether your drive is an SSD or HDD and adjusts mount options accordingly
- **ext4**, the classic stable Linux filesystem. Widely supported, no snapshot capability
- **XFS**, high performance, good for large files, no snapshots

What is a Filesystem?

A filesystem is the method by which an operating system organises and stores data on a drive. Linux supports multiple filesystems, Btrfs, ext4, XFS among others. Windows uses **NTFS**. Linux can read NTFS drives, but installing Linux itself on an NTFS partition is not supported or recommended.

Select **Btrfs** and click **Next**.

2.4.8. Desktop Environment

Desktop Environment vs. Window Manager

A **Desktop Environment** (DE) is a complete, integrated desktop experience — file manager, settings app, notification tray, lock screen, and more, all packaged together. Think of it as a fully furnished house. Examples: KDE Plasma, GNOME, XFCE.

A **Window Manager** (WM) controls only window behaviour — resizing, tiling, keyboard shortcuts. Think of it as the bare structure of a house with no furniture; you build everything yourself. Examples: Hyprland, niri, i3. Every DE ships with a WM already included — KDE uses KWin, GNOME uses Mutter.

For this guide, choose a Desktop Environment. Tiling window managers like Hyprland are powerful but not appropriate for a first Linux install.

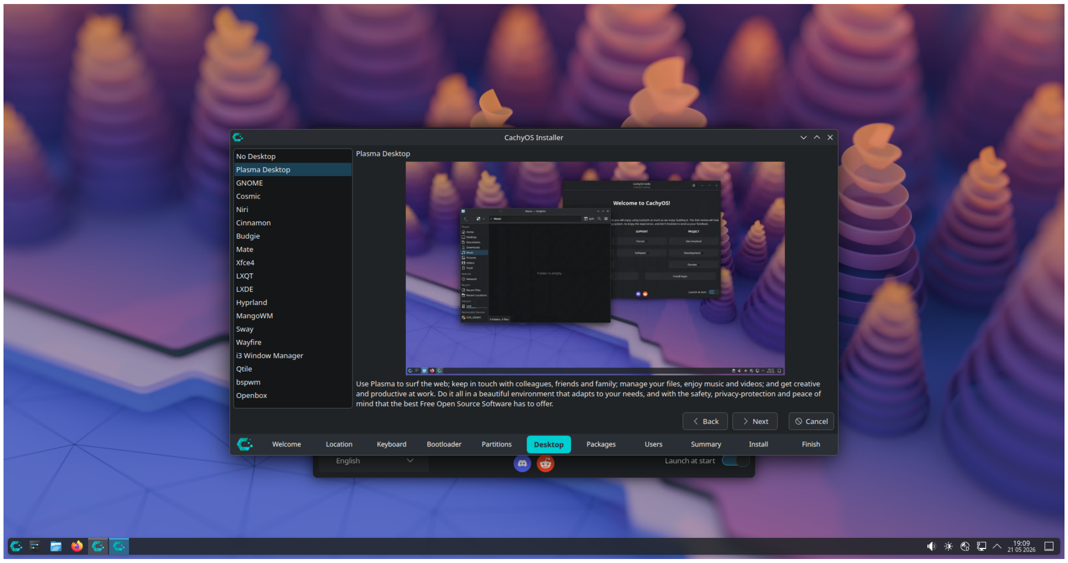
The available options for beginners:

- **KDE Plasma** (recommended): modern, feature-rich, highly customizable. Workflow is similar to Windows. The default choice for this guide
- **GNOME**: minimal, opinionated, clean. Closer in feel to macOS than Windows. A different workflow that takes adjustment
- **XFCE**: lightweight and efficient. Recommended only for older hardware with limited RAM

Note

There is no objectively best Desktop Environment. The difference is workflow. KDE is the starting point for most beginners due to its similarity to Windows and its depth of customization options. You are not locked in; CachyOS supports multiple DEs installed simultaneously, and you can switch later.

Select **KDE Plasma** and click **Next**.



Desktop Environment selection in Calamares

2.4.9. Packages

Calamares will present a package selection screen. Ensure the default selections are checked, these include the base KDE packages and essential system components. Do not uncheck anything unless you know what you are removing. Click **Next**.

2.4.10. Users

Fill in the following fields:

- **Your name**: display name, can be anything
- **Username**: your login name, lowercase, no spaces
- **Computer name**: the hostname of your machine on the network
- **Password**: used for login and every `sudo` command. Do not forget this

> **Warning**
>
> Your password is required for every administrative action on the system. There is no password recovery prompt like Windows — if you forget it, recovery requires booting into a live environment and manually resetting it. Write it down somewhere safe.

2.4.11. Summary and Installation

Calamares will display a full summary of your chosen configuration, language, timezone, bootloader, filesystem, desktop, and username. Review everything carefully. If anything is incorrect, use the **Back** button to correct it.

When satisfied, click **Install**. The installer will begin writing to disk. Depending on your internet connection and drive speed, this takes between **30 minutes and 2 hours**.

> **During Installation**
>
> Do not close the installer, put the machine to sleep, or disconnect from the internet while installation is in progress. An interrupted install can leave the drive in an unbootable state.

What Happens During Installation

At around 30%, the copying of files is completed, this takes the longest. After that, the installer creates users, removes live-system-specific packages, installs additional packages, removes language packs and drivers not needed for your hardware, and sets up the bootloader. On a network install, time depends entirely on your internet speed and mirror proximity.

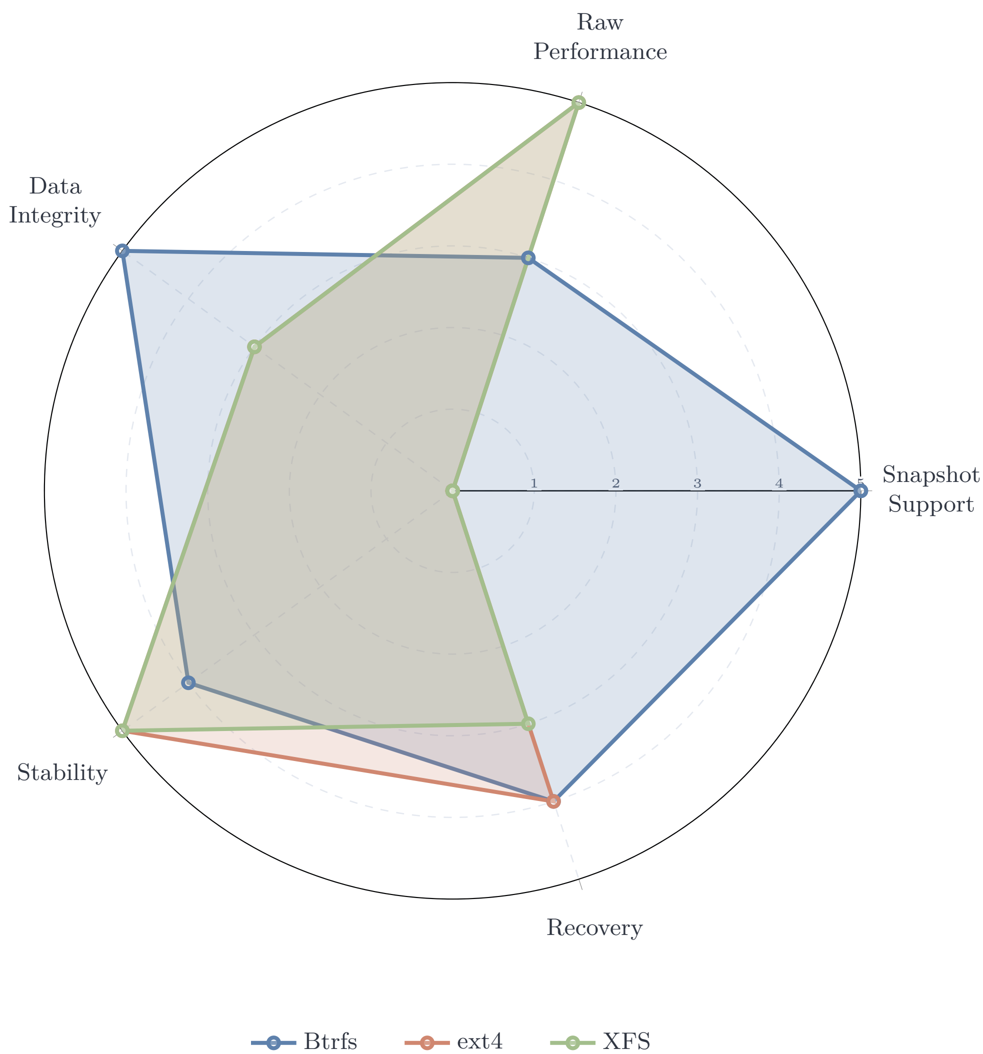
> **Progress Stages**
>
> The progress bar is not linear. It will appear to stall. According to the official CachyOS FAQ, stalling at 33% is almost always a network or mirror issue, not a system failure. Wait at least five minutes before concluding something is wrong.

If Installation Fails

If an installation fails, the installer may not properly unmount partitions, meaning continuous errors can occur if you attempt to reinstall immediately. Always reboot back into the live ISO before trying again.

After Installation Completes

When the installer finishes, Calamares will prompt you to restart. Remove the USB drive when instructed and press ⏎. Your system will boot into the **Limine** bootloader menu, a clean interface showing your installed operating systems. Select **CachyOS** and press ⏎.
You are now running CachyOS. Section 3 covers what to do immediately after first boot.



Filesystem	Snapshot	Performance	Integrity	Stability	Recovery
Btrfs	5	3	5	4	4
ext4	0	5	3	5	4
XFS	0	5	3	5	3

Qualitative comparison based on Linux Journal, Linuxano, and VPS.DO filesystem analyses (2025–2026).
Scores reflect desktop gaming use case priorities.

2.5. Post-Install Checks

The first boot into a fresh CachyOS install is not the time to start installing games. Before anything else, the system needs to be verified, updated, and confirmed healthy. Skipping these steps is how systems end up in broken states two weeks later.

2.5.1. Update the System

Open a terminal. On KDE Plasma, press META or search for **Konsole** in the application launcher. Run:

```
sudo pacman -Syu
```

This synchronizes the package database and upgrades all installed packages. On a fresh online install the list will be short. On an offline install, expect a significantly larger download.

Warning

Never run `pacman -Sy` or `pacman -Sy package` without the `u` flag. A partial upgrade on Arch-based systems introduces dependency mismatches that can break the entire system. Always use `-Syu`.

2.5.2. Verify paru; Your AUR Helper

The **Arch User Repository** (AUR) is a community-maintained repository of build scripts for software not available in the official Arch repositories. It is one of the biggest practical advantages of using an Arch-based system. Nearly any software you can think of has an AUR package.

To use the AUR, you need an **AUR helper**. CachyOS ships with `paru`, a feature-rich AUR helper written in Rust, installed by default. Confirm it is present:

```
paru --version
```

If it prints a version number, `paru` is ready. If the terminal returns `command not found`, install it manually:

```
sudo pacman -S --needed base-devel
git clone https://aur.archlinux.org/paru.git
cd paru
makepkg -si
```

paru vs pacman

`paru` is a wrapper around `pacman`. It handles everything `pacman` does, plus searches and builds AUR packages. The syntax is identical, anywhere you would use `pacman`, you can use `paru` instead. According to the official CachyOS FAQ, when AUR packages are installed, use `paru -Syu` for updates instead of `pacman -Syu` to ensure AUR packages are upgraded as well.

A Note on Graphical Package Managers

The official CachyOS FAQ explicitly warns against using **Pamac** for system package management. It is known to improperly handle certain package management tasks, such as corrupting system keyrings, leading to PGP signature errors that prevent future updates. Use `pacman` or `paru` from the terminal for system updates.

2.5.3. Enable Multilib

Multilib is a repository that provides 32-bit libraries on a 64-bit system. It is required for Steam, many games, and Wine. On CachyOS it is typically enabled by default, but verify it is active by opening the 🗁 /etc/pacman.conf:

```
sudo nano /etc/pacman.conf
```

Scroll down and confirm the following lines are present and **uncommented** (no `#` at the start):

```
[multilib]
Include = /etc/pacman.d/mirrorlist
```

If they are commented out, remove the `#` characters, save with Ctrl+O, and exit with Ctrl+X. Then sync:

```
sudo pacman -Syu
```

2.5.4. Verify GPU Drivers

Before launching any games, confirm your GPU drivers are correctly loaded. Run:

```
glxinfo | grep "OpenGL renderer"
```

The output should display your GPU name, for example:

```
OpenGL renderer string: NVIDIA GeForce RTX 3070/PCIe/SSE2
```

Warning

If the output shows `llvmpipe` or `softpipe`, the correct GPU drivers are not loaded. These are software renderers. Games will run at a fraction of their normal performance or not at all. Do not proceed to gaming setup until this is resolved.

If `glxinfo` is not found, install it with:

```
sudo pacman -S mesa-utils
```

2.5.5. Check Critical systemd Services

Confirm that essential services are running correctly:

```
systemctl --failed
```

A healthy system returns an empty list. If any services appear as `failed`, investigate them individually:

```
systemctl status service-name
```

NetworkManager

The most common failed service on a fresh install is `NetworkManager`. If your network is working, it is running. This is sometimes a display artefact. Verify with:

```
systemctl status NetworkManager
```

It should show `active (running)`.

With the system updated, `paru` confirmed, multilib enabled, GPU drivers verified, and services healthy, the system is ready. Section 3 covers the first boot experience and initial desktop configuration.

3. First Boot

You have successfully installed CachyOS. This section walks through the first boot sequence, the login screen, and getting comfortable with your new desktop environment before touching anything system-critical.

3.1. The Limine Screen

After removing the USB and rebooting, your system will present the **Limine** bootloader menu. Use ↑ and ↓ to navigate to **CachyOS** and press ↵.

3.2. The Login Screen

You will be greeted by a graphical login screen. This is the **display manager** — a program that provides a graphical login interface, handles user authentication, and launches your chosen desktop environment.

What is a Display Manager?

The display manager is the first graphical program that runs after boot. It starts the graphical environment, presents a login screen, authenticates the user, and then hands control to the desktop environment — in this case, KDE Plasma. CachyOS uses **SDDM** (Simple Desktop Display Manager) by default.

Type the password you set during installation and press ↵.

3.3. Welcome to Linux

CachyOS will greet you with the **CachyOS Hello** window — a graphical tool for post-install configuration, system tweaks, and package installation. You will return to it later. For now, set it aside.

3.4. Making Yourself at Home

Before doing anything system-critical, spend a few minutes becoming comfortable with the desktop. This is not wasted time — familiarity with the UI makes everything that follows easier.

3.4.1. Changing the Wallpaper

1. Right-click on the desktop
2. Select `Desktop and Wallpaper` or press `Ctrl+Shift+D`
3. Choose a preloaded wallpaper, or add your own using the **Add** button

3.4.2. Customising the Panel

1. Right-click the bottom panel
2. Select `Show Panel Configuration`
3. Adjust layout, widgets, and position to your preference

3.4.3. Things Worth Trying

- Verify all keyboard shortcuts and keys are working. Customise them via System Settings ⟫ Shortcuts
- Take a screenshot with PrtSc
- Explore System Settings ⟫ Display and Monitor to configure refresh rate and scaling
- Explore System Settings ⟫ Appearance — KDE ships with thousands of community-made themes, icon packs, and window decorations

3.5. The Terminal

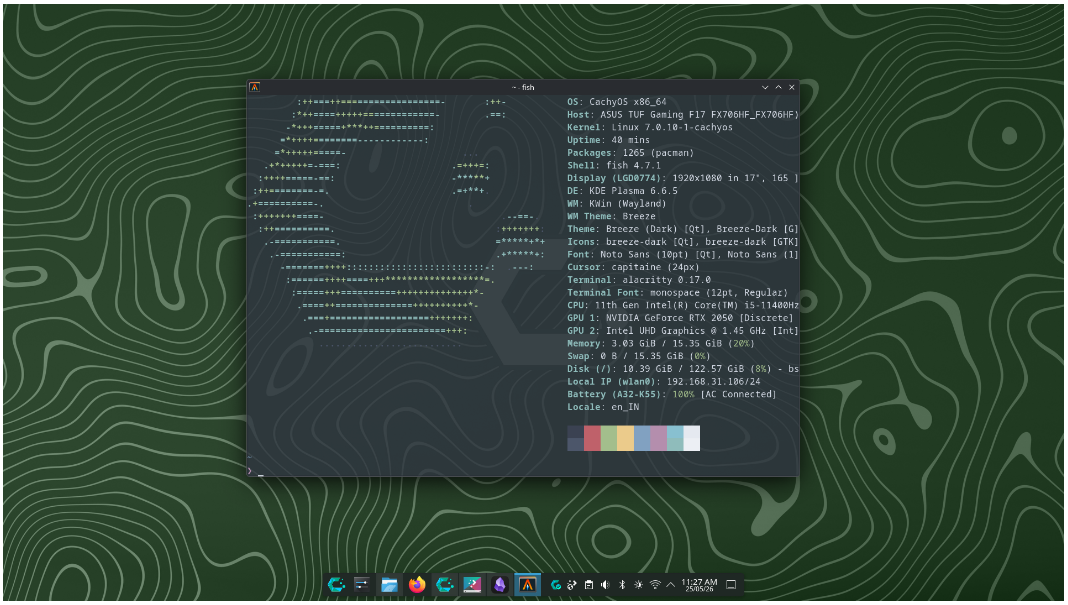
On Linux, the terminal is not optional. It is the primary tool for system administration, package management, and troubleshooting. A **terminal emulator** is the graphical application that provides a window for text input. Inside it runs a **shell**, the actual command interpreter that reads your input and instructs the OS what to do.

Terminal vs. Shell

The **terminal** is the window. The **shell** is the program running inside it. Common shells include `bash`, `zsh`, and `fish`. CachyOS uses **Fish** by default — a user-friendly shell with autocompletion and syntax highlighting out of the box. The terminal emulator bundled with CachyOS is **Alacritty** — a GPU-accelerated terminal built for speed.

To open Alacritty, search for it in the application launcher. Pin it to the taskbar, you will use it constantly. When Alacritty opens, you will see your system information displayed automatically. This is **fastfetch** — a pre-configured tool that prints a formatted system summary on terminal launch. Run it again at any time with:

```
fastfetch
```



Alacritty terminal showing fastfetch system information output

Why fastfetch matters

When asking for help on any Linux forum or subreddit, always include a screenshot of your fastfetch output. It tells others your distro, kernel version, DE, GPU, and RAM at a glance — essential context for troubleshooting.

4. File System

Linux organises all files under a single unified directory tree rooted at `/`. Understanding this structure is essential — knowing where configuration files, logs, and game data live will save significant time when troubleshooting. This section covers the official filesystem standard, the directory layout, and how to navigate it both graphically and through the terminal.

4.1. The Filesystem Hierarchy Standard

Every file and directory on a Linux system lives under a single root directory: `/`. This structure is defined by the **Filesystem Hierarchy Standard** (FHS) — a specification maintained by the Linux Foundation that defines the directory layout and contents of Unix-like operating systems.

> *"This standard consists of a set of requirements and guidelines for file and directory placement under UNIX-like operating systems. The guidelines are intended to support interoperability of applications, system administration tools, development tools, and scripts as well as greater uniformity of these systems."*
>
> — Filesystem Hierarchy Standard 3.0, Linux Foundation

All files and directories appear under the root directory `/`, even if they are stored on different physical or virtual devices. This is fundamentally different from Windows, which assigns a drive letter (`C:`, `D:`) to each storage device. On Linux, everything is one unified tree.

4.2. The Directory Tree

```
/
├── bin/        # essential binaries (ls, cat, cp...)
├── boot/       # kernel, initramfs, bootloader
├── dev/        # device files (disks, USB, TTY...)
├── etc/        # system-wide configuration files
├── home/       # user home directories
│   └── user/
│       ├── Documents/
│       ├── Downloads/
│       └── ...
├── lib/        # essential libraries for /bin
├── media/      # auto-mounted removable drives
├── mnt/        # manual mount points
├── opt/        # optional/third-party software
├── proc/       # virtual fs: live kernel & process info
├── root/       # home directory of the root user
├── run/        # runtime data (PIDs, sockets)
├── srv/        # data for services (web, FTP...)
├── sys/        # virtual fs: hardware & kernel info
├── tmp/        # temporary files (cleared on reboot)
├── usr/        # user utilities and applications
│   ├── bin/    # most user commands live here
│   ├── lib/    # libraries for /usr/bin
│   ├── local/  # locally compiled software
│   └── share/  # shared data (icons, docs, themes)
└── var/        # variable data
    ├── log/    # system logs
    ├── cache/  # application cache
    └── spool/  # print and mail queues
```

Structure defined by FHS 3.0, Linux Foundation. Identical across all FHS-compliant distributions.

FHS 3.0

The most recent version of the FHS, version 3.0, was published on March 18, 2015, and is maintained by the Linux Foundation. Most Linux distributions are FHS-compliant, with the notable exception of NixOS, which uses a unique hierarchy built around the Nix package manager.

4.3. Key Directories for Gamers

Most of what you will interact with as a gamer lives in a small subset of these directories:

Path	Relevance
`/home/username/`	Your personal files, game saves, configs
`/home/username/.local/share/Steam`	Steam installation and game library
`/home/username/.steam`	Steam runtime and Proton data
`/etc/`	System-wide configuration files
`/var/log/`	System logs for troubleshooting
`/tmp/`	Temporary files, cleared on reboot

Hidden Files

Files and directories beginning with a `.` (dot) are hidden by default — `.steam`, `.config`, `.local`. They are not shown by `ls` unless you use `ls -a`. In Dolphin (KDE's file manager), press Ctrl+H to toggle their visibility.

4.4. Navigating with the Terminal

Open Alacritty and try the following in order. This is not a reference — it is a short walkthrough to build muscle memory.

Start by confirming where you are:

```
pwd
```

This will print `/home/username` — your home directory is the default working directory every time a terminal opens. Now list everything in it:

```
ls -a
```

Your output will look similar to this:

```
.cache      .config      .local        .pki
.var        Desktop      Documents     Downloads
Music       Pictures     Projects      Public
Templates   Videos       .bash_logout  .bash_profile
.bashrc     .gtkrc-2.0   .zshrc
```

Entries beginning with `.` are hidden files and directories. Configuration files, shell profiles, and application data. They are invisible by default in both `ls` (without `-a`) and in Dolphin (without Ctrl+H).

Now navigate into a subdirectory:

```
cd Pictures/Screenshots
```

Tab Completion

While typing any path, press Tab to autocomplete. If multiple matches exist, press Tab twice to see them all. Make this a habit — it prevents typos and is significantly faster than typing full paths.

Once inside `Pictures/Screenshots`, list its contents:

```
ls
```

To open a file directly from the terminal, type its name. For an image file, your default viewer will launch:

```
xdg-open screenshot.png
```

To return home at any point:

```
cd ~
```

4.5. Navigating with Dolphin

Dolphin is KDE's file manager and a capable graphical alternative to the terminal for everyday file operations. Open it from the taskbar or application launcher and spend a few minutes exploring — copy, paste, rename, and move files the same way you would on Windows.

Useful shortcuts:

- Ctrl+H — toggle hidden files and folders
- F4 — open an embedded terminal panel at the current directory
- Ctrl+L — focus the path bar for direct path entry
- Alt+Left / Alt+Right — navigate back and forward

F4 is Underrated

Pressing F4 in Dolphin opens a terminal panel locked to the directory you are currently browsing. This is the fastest way to drop into a terminal at a specific location without typing `cd` repeatedly.

4.5.1. Useful Commands

The four commands you will use constantly:

```
pwd          # print current working directory
ls           # list directory contents
ls -a        # list including hidden files
cd Pictures  # change into the Pictures directory
cd ~         # return to home directory
```

4.6. Launching Apps from the Terminal

Every application on Linux can be launched by typing its name in the terminal:

```
firefox     # launch Firefox and see its logs
steam       # launch Steam with full log output
vlc         # launch VLC
```

This is more than a curiosity — when an application crashes or behaves unexpectedly, launching it from the terminal prints its log output in real time. This output is often the fastest way to diagnose what went wrong, and is what Linux forums and subreddits will ask you to provide when requesting help.

5. Terminal Basics

The terminal is the most direct interface between you and the operating system. This section covers command structure, essential commands, keyboard shortcuts, and piping — everything needed to operate confidently from the command line before moving into package management and system configuration.

5.1. Why the Terminal?

The terminal is the most direct interface between you and the operating system. While graphical tools exist for most tasks, the terminal is faster, scriptable, and always available — even when the desktop environment breaks. On a rolling-release distribution like CachyOS, knowing your way around the terminal is not optional.

> *"The shell is a command interpreter and programming language that provides the user interface to a Linux system. It interprets commands typed by the user and translates them into instructions the kernel can execute."*
>
> — GNU Bash Manual, gnu.org

5.2. Anatomy of a Command

Every command follows the same structure:

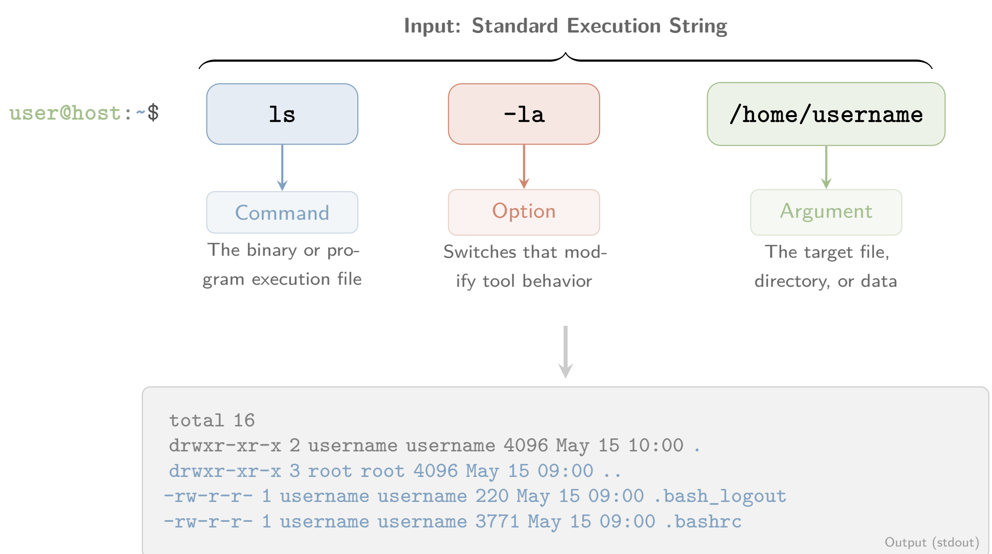
```
command [options] [arguments]
```

For example:

```
ls -la /home/username
```

Where `ls` is the command, `-la` are options (list all files in long format), and 🗀 `/home/username` is the argument (the target directory). Options beginning with a single dash (`-`) are short flags; those with double dashes (`-`) are long-form equivalents:

```
ls -a
ls --all    # identical result
```



5.3. Essential Commands

```
# Navigation
pwd                     # print current directory
cd /etc                 # change to /etc
cd ..                   # go up one directory
cd ~                    # go to home directory

# Files and directories
ls -la                  # list all files with details
mkdir myfolder          # create a directory
rm file.txt             # delete a file
rm -rf myfolder         # delete a directory recursively
cp file.txt backup/     # copy a file
mv file.txt docs/       # move a file
cat file.txt            # print file contents

# System
sudo command            # run command as administrator
man ls                  # open the manual for ls
which firefox           # find where a binary lives
echo $SHELL             # print your current shell
```

Warning

`rm -rf` is permanent and irreversible. There is no trash bin. Double-check the path before executing. Running `rm -rf /` as root deletes the entire filesystem. According to the Arch Wiki Security guide, never run commands from the internet without reading and understanding them first.

5.4. Reading Manual Pages

Every standard Linux command has a manual page:

```
man pacman
```

Navigate with ↑/↓, search with / followed by a search term, and quit with q.

5.5. Useful Shortcuts

Shortcut	Action
Ctrl+C	Kill the running process
Ctrl+Z	Suspend the running process
Ctrl+D	Exit the current shell session
Ctrl+L	Clear the terminal screen
Tab	Autocomplete command or path
↑	Cycle through command history
Ctrl+R	Search command history interactively

5.6. Pipes and Redirection

Commands can be chained together using pipes (`|`) to pass output from one command into another:

```
# Find all running processes and search for steam
ps aux | grep steam

# Count lines in a file
wc -l /var/log/pacman.log

# Save command output to a file
journalctl -b > boot_log.txt
```

A Note on Fish Shell

CachyOS uses **Fish** as the default shell. Fish differs from `bash` in syntax for scripting (variables use `set` instead of `=`, for example), but for interactive use and the commands in this guide, the difference is invisible. If you copy `bash` scripts from the internet and they fail in Fish, run them explicitly with `bash script.sh`.

6. Package Manager Basics

On Linux, software does not come from scattered websites and `.exe` installers. It comes from **repositories** — curated, version-controlled collections of packages maintained by the distribution team and the community. The package manager is the tool that interfaces with these repositories: installing, upgrading, and removing software with a single command, resolving dependencies automatically, and keeping the entire system consistent. On CachyOS, that tool is `pacman`, supplemented by `paru` for the AUR.

6.1. What is a Package Manager?

A package manager is the tool that installs, upgrades, and removes software on your system. Unlike Windows, where software is scattered across manufacturer websites and installers, Linux centralises everything through repositories — curated collections of software maintained by the distribution team and the community.

> *"Pacman is a package management utility that tracks installed packages on a Linux system. It features dependency support, package groups, install and uninstall scripts, and the ability to sync your local machine with a remote repository to automatically upgrade packages."*
>
> — pacman man page, archlinux.org

6.2. Where Packages Come From

On CachyOS there are three sources, in order of preference:

1. **Official repositories** — packages maintained and tested by the Arch Linux team and CachyOS maintainers. Installed via `pacman`. Always prefer this source
2. **AUR** (Arch User Repository) — community-maintained build scripts for software not in official repos. Installed via `paru`. Verify the PKGBUILD before installing
3. **Build from source** — compile directly from a project's source code using `makepkg`. Use only when the above two options are unavailable

AUR Safety

The AUR is community-maintained — anyone can submit a package. According to the CachyOS FAQ, always read the PKGBUILD before installing an AUR package, and check the comment section for reported issues. Some packages are maintained by the software developers themselves (Google Chrome, Microsoft VS Code), which are generally safe.

Table 1: Overview of CachyOS Package Sources and Tools

Source	Tool	Maintained by	Safety	Speed
Official Repos	`pacman`	Arch/CachyOS team	Highest	Fastest
AUR	`paru`	Community	Moderate	Fast
AUR	`yay`	Community	Moderate	Fast
Flatpak	`flatpak`	Flathub/Developers	High (Sandboxed)	Moderate
Source	`makepkg`	You	Verify yourself	Slowest

Note: While `pacman` remains the core manager, `paru` and `yay` act as automated wrappers for the AUR. `Flatpak` provides an alternative, sandboxed layer, for desktop applications.

6.3. pacman Cheat Sheet

```
# Update system
sudo pacman -Syu

# Install a package
sudo pacman -S vlc

# Search for a package
pacman -Ss vlc

# Remove a package and its dependencies
sudo pacman -Rns vlc

# List installed packages
pacman -Q

# Search installed packages
pacman -Qs vlc

# Clean old package cache
sudo pacman -Sc

# Detailed info on installed package
pacman -Qi vlc

# Detailed info from remote repo
pacman -Si vlc

# Remove orphaned packages
sudo pacman -Qdtq | pacman -Rns -

# Clear entire cache (aggressive)
sudo pacman -Scc
```

6.4. Installing from the AUR with paru

`paru` uses identical syntax to `pacman`:

```
# Install from AUR
paru -S spotify

# Update everything including AUR packages
paru -Syu

# Search AUR
paru -Ss spotify
```

Warning

Avoid **Pamac** for system updates. The official CachyOS FAQ explicitly warns that Pamac is known to corrupt system keyrings on rolling-release systems, leading to PGP signature errors that block future updates. Use `pacman` or `paru` from the terminal.

6.5. Building from Source

For software unavailable in official repos or the AUR, `makepkg` compiles directly from source. This requires `base-devel`:

```
# Install build tools
sudo pacman -S --needed base-devel git

# Clone the source repository
git clone https://github.com/somedev/someproject
cd someproject

# Build and install
makepkg -si
```

When to Build from Source

Building from source is rarely necessary on CachyOS. The AUR covers the vast majority of software not in official repos. Resort to source builds only when no maintained AUR package exists or the AUR version is severely outdated.

7. Permissions

Every file and directory on a Linux system carries a set of permissions that controls who can read it, write to it, or execute it. Understanding permissions is not optional — you will encounter them constantly when configuring the system, running scripts, and troubleshooting access errors.

7.1. The Three Permission Groups

> *"chmod changes the file mode bits of each given file according to mode, which can be either a symbolic representation of changes to make, or an octal number representing the bit pattern for the new mode bits."*
>
> — GNU Coreutils Manual, gnu.org

According to the GNU Coreutils documentation, every file on a Linux system has three permission groups:

- **Owner** (`u`): the user who created the file. Has the most control by default
- **Group** (`g`): every user belongs to one or more groups. When a file is created, it is assigned to the owner's primary group. Users in that group share the group permissions
- **Other** (`o`): every other user on the system who is neither the owner nor in the file's group

Each group can hold three permissions:

- **r** — read: view file contents or list directory contents
- **w** — write: modify a file or create/delete files inside a directory
- **x** — execute: run a file as a program, or enter a directory

7.2. Reading the Permission String

Run the following in your terminal:

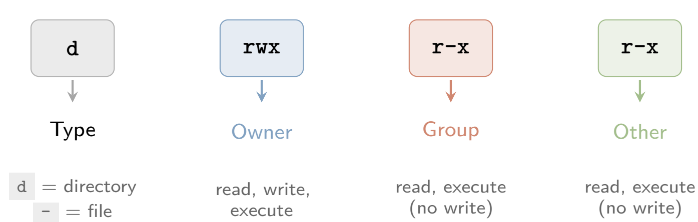
```
ls -l
```

```
drwxr-xr-x
│ │  │  │
│ │  │  └── Other permissions (r-x)
│ │  └───── Group permissions (r-x)
│ └──────── Owner permissions (rwx)
└────────── Directory (d) or file (-)
```

Each entry will show a permission string like this:

```
drwxr-xr-x  username  26 May 14:45  Downloads
```

The first field, `drwxr-xr-x`, is the permission string. Here is how to read it:



So `drwxr-xr-x` means: this is a **directory**; the **owner** can read, write, and enter it; the **group** can read and enter it but not modify its contents; **others** have the same access as the group.

Execute on Directories

The `x` permission on a directory does not mean running it as a program — it means permission to *enter* the directory and access files inside it. Without `x`, you cannot `cd` into a directory even if you can read it.

7.3. Modifying Permissions with `chmod`

`chmod` (change mode) is the command used to modify file and directory permissions. According to the GNU Coreutils manual, only the file's owner or a process with appropriate privileges (i.e. `sudo`) is permitted to change its mode bits.

Syntax:

```
chmod [mode] [file]
```

There are two ways to express the mode: **symbolic** and **numeric**.

7.3.1. Symbolic Mode

Symbolic mode uses letters to specify who is affected and what changes:

Symbol	Meaning
`u`	Owner (user)
`g`	Group
`o`	Other
`a`	All (owner, group, and other)
`+`	Add permission
`-`	Remove permission
`=`	Set exact permission

Examples:

```
# Give the owner execute permission
chmod u+x file.txt

# Give others read permission
chmod o+r file.txt

# Remove write permission from group
chmod g-w file.txt

# Give everyone read and execute, remove write
chmod a=rx file.txt
```

7.3.2. Numeric Mode

Numeric (octal) mode represents permissions as numbers. According to the Debian coreutils man page, each permission type maps to a value:

Permission	Symbol	Value
Read	`r`	4
Write	`w`	2
Execute	`x`	1
None	`-`	0

Add the values together for each group to get a three-digit number:

```
rwx = 4+2+1 = 7
r-x = 4+0+1 = 5
r-- = 4+0+0 = 4
rw- = 4+2+0 = 6
```

So `drwxr-xr-x` from the earlier example becomes `755`:

```
chmod 755 file.txt    # rwx for owner, r-x for group and other
chmod 644 file.txt    # rw- for owner, r-- for group and other
```

644 and 755 — The Two You Will See Everywhere

644 (`rw-r-r-`) is the standard permission for files — the owner can read and write, everyone else can only read. **755** (`rwxr-xr-x`) is the standard for directories and executables — the owner has full access, everyone else can read and execute. These two appear constantly in Linux documentation and forum answers.

Numeric Mode is Not Optional Knowledge

Representations like `755` and `644` appear throughout Linux documentation, forum answers, and configuration guides. Even if you prefer symbolic mode day-to-day, you need to be able to read and write numeric permissions fluently.

7.4. sudo — Elevated Permissions

Some files and directories are owned by `root`, the system administrator account. Modifying them requires elevated privileges, granted by the `sudo` command:

```
sudo chmod 755 /etc/somefile
```

Warning

Do not run `chmod -R 777 /` or grant world-write permissions to system directories. `777` gives every user on the system full read, write, and execute access to a file — a serious security risk. According to the Arch Wiki Security guide, file permissions are a primary mechanism of Linux system security. Use the minimum permissions necessary.

8. Setting Up NVIDIA Drivers and Gaming Packages

Drivers are the software layer between the operating system and your GPU. Without the correct driver loaded, the GPU runs on a generic fallback renderer — functional, but completely unsuitable for gaming. This section covers driver installation for both AMD and NVIDIA GPUs, and explains the driver landscape clearly so you can identify exactly what your hardware needs.

8.1. AMD GPUs

If you have an AMD GPU, the situation is straightforward.

> *The `amdgpu` kernel driver is the open-source AMD GPU driver maintained upstream in the Linux kernel. It supports all GCN and RDNA architecture cards and is included in the kernel by default — no separate installation is required on a modern distribution like CachyOS.*
>
> — Arch Wiki: AMDGPU

Verify that the `amdgpu` driver is active:

```
lspci -k | grep -A 3 VGA
```

`lspci` lists all PCI devices. The `-k` flag shows which kernel driver is in use for each device. `grep -A 3 VGA` filters the output to show only the GPU entry and the three lines below it — where the driver name appears. The output should include:

```
Kernel driver in use: amdgpu
```

If the driver is not loaded, install the required packages manually:

```
sudo pacman -S mesa vulkan-radeon lib32-mesa lib32-vulkan-radeon
```

`mesa` provides the OpenGL implementation. `vulkan-radeon` is the Vulkan driver (required by most modern games and Proton). The `lib32-` prefixed variants are the 32-bit equivalents, required by Steam and many games.

AMD on Linux — The Short Version

AMD GPUs work out of the box on Linux because AMD has contributed their driver directly to the Linux kernel since 2014. There is no proprietary driver to install, no conflicts to manage, and no risk of a kernel update breaking your graphics. For gaming on Linux, AMD is the low-friction choice.

8.2. NVIDIA GPUs

NVIDIA's driver situation on Linux is more complex. CachyOS ships with the open-source **Nouveau** driver enabled by default, but Nouveau does not support hardware acceleration for modern NVIDIA cards adequately — the Arch Wiki describes it as suitable for basic desktop use only. For gaming, the proprietary NVIDIA driver is required.

NVIDIA Driver Landscape (2025–2026)

NVIDIA maintains multiple driver branches simultaneously. As of December 2025, Arch Linux officially announced that the NVIDIA 590 driver dropped support for Pascal (GTX 10xx) and older architectures. Turing (RTX 20xx) and newer now use `nvidia-open` — NVIDIA's partially open-source kernel module. Pascal and older must use legacy driver branches from the AUR.

8.2.1. Identifying Your GPU Architecture

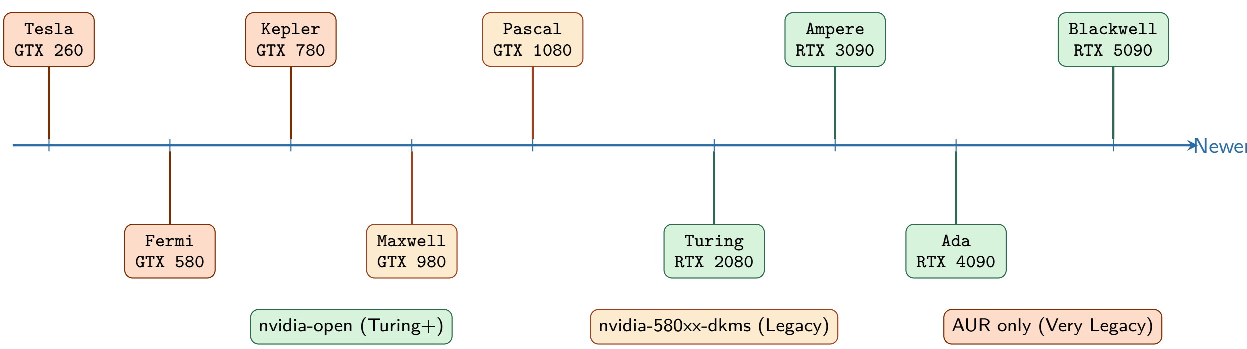
Before installing any driver, identify your GPU and its architecture:

```
lspci | grep -i nvidia
```

Architecture	Example GPUs	Driver
Blackwell	RTX 5060, 5070, 5080, 5090	`nvidia-open`
Ada Lovelace	RTX 4060, 4070, 4080, 4090	`nvidia-open`
Ampere	RTX 3050–3090, RTX 2050	`nvidia-open`
Turing	GTX 1650, 1660, RTX 2060–2080 Ti	`nvidia-open`*
Volta	Titan V	`nvidia-580xx-dkms`
Pascal	GTX 1050–1080 Ti	`nvidia-580xx-dkms`
Maxwell	GTX 750 Ti, 960–980 Ti	`nvidia-580xx-dkms`
Kepler	GTX 660, 680, 780 Ti	`nvidia-470xx-dkms`
Fermi	GTX 460, 560 Ti, 580	`nvidia-390xx-dkms`
Tesla	GTX 260, GTX 280	`nvidia-340xx-dkms`

* Some Turing cards may need `nvidia-580xx-dkms` for power management — see note below.

This prints your GPU model name. Use the timeline below to locate your architecture and identify the correct driver branch:



Driver branches per architecture. Source: Arch Linux News, December 2025 and wiki.cachyos.org

8.2.2. Installing NVIDIA Drivers

Turing, Ampere, Ada Lovelace, Blackwell (RTX 20xx and newer):

```
sudo pacman -S linux-cachyos-nvidia-open
sudo pacman -S nvidia-utils lib32-nvidia-utils nvidia-settings
```

`linux-cachyos-nvidia-open` is the CachyOS-patched NVIDIA open kernel module, pre-built for the CachyOS kernel. `nvidia-utils` provides user-space utilities including `nvidia-smi`. `lib32-nvidia-utils` provides 32-bit support required by Steam. `nvidia-settings` is the graphical driver configuration tool.

Turing Power Management

Some Turing cards (RTX 20xx) experience power management issues with `nvidia-open`. If your system fails to suspend, resume, or experiences instability, switch to the legacy proprietary branch instead:

```
paru -S nvidia-580xx-dkms
```

Maxwell, Pascal (GTX 900, GTX 10xx):

```
paru -S nvidia-580xx-dkms
```

`nvidia-580xx-dkms` is the last driver branch supporting Maxwell and Pascal. It is in the AUR rather than official repos because, as of December 2025, Arch Linux dropped Pascal and older from the official NVIDIA packages. `dkms` means the driver rebuilds itself automatically against new kernels.

Kepler (GTX 600, GTX 700):

```
paru -S nvidia-470xx-dkms
```

Fermi (GTX 400, GTX 500):

```
paru -S nvidia-390xx-dkms
```

Tesla (GTX 200):

```
paru -S nvidia-340xx-dkms
```

Reboot Required

After installing any NVIDIA driver, reboot before proceeding. The driver is not active until the system restarts and loads the new kernel module. Running games or verifying the driver before rebooting will produce incorrect results.

8.2.3. Verifying the NVIDIA Driver

After rebooting, confirm the driver is loaded correctly:

```
nvidia-smi
```

`nvidia-smi` (System Management Interface) is a tool provided by `nvidia-utils` that queries the NVIDIA driver directly. A successful output displays your GPU model, the loaded driver version, VRAM usage, and running processes. If the command returns `command not found` or an error, the driver did not load correctly — check `journalctl -b` for error messages related to `nvidia`.

Double-Check the Driver Version

Confirm the driver version shown by `nvidia-smi` matches the package you installed. A mismatch between the kernel module version and `nvidia-utils` version causes errors. If they differ, run `sudo pacman -Syu` to synchronise everything.